

Task 17 —: Build a Cloud Operations Monitoring and Incident Exercise

Intern: Nidhi Pandey

Program: Evolixa AWS Cloud Internship

Date: 2026-09-05

1. Approach and Architecture

The objective of this task was to create a basic operational workflow for detecting and responding to problems affecting an EC2-hosted application. The scenario involved a fictional application that begins experiencing intermittent failures, and the operations team needed a simple process for detecting the problem, investigating it, and documenting the response.

My approach was to simulate a real-world incident from detection through resolution. I started by defining the application's normal operating state and identifying useful monitoring indicators. I then configured CloudWatch monitoring to detect anomalies, created a controlled test condition to simulate a failure, observed the symptoms, investigated the root cause, applied corrective action, and verified recovery. Finally, I documented the entire incident timeline, root cause analysis, and created an incident-response runbook.

2. Implementation Steps

Step 1 – Define the Application's Normal Operating State: I defined the normal operating state of the application. In a healthy state, the application is running, CPU usage is below

70%, memory usage is below 80%, the website is accessible with HTTP 200 status, and error logs show no errors. Response time is under 500ms.

Step 2 – Identify Useful EC2/Server Indicators: I identified key indicators to monitor for server health. These included CPU Utilization, Memory Usage, Disk Usage, HTTP Status, Error Logs, and Response Time. Each indicator provides valuable information about the server's performance and health.

Step 3 – Configure Appropriate Monitoring Where Available: I created a CloudWatch alarm named task17-cpu-alarm that triggers when CPU utilization exceeds 80% for 5 minutes. I also created an SNS topic called task17_alarm_topic to send email notifications when the alarm fires. This ensures the operations team is immediately alerted to potential issues.

Step 4 – Create a Controlled Test Condition That Represents an Application/Server Issue: To simulate a failure, I used the stress command on the EC2 instance to spike CPU usage to 100%. This created a controlled test condition that mimics a real-world application issue.

Step 5 – Observe the Available Indicators/Logs: When the stress command was running, I observed the indicators. The CloudWatch alarm triggered, CPU usage reached 100%, and the website became slow with increased response time.

Step 6 – Identify the Symptoms: The symptoms of the issue were clear: CPU usage was at 100%, response time increased to 5000ms, HTTP status returned 500/504 errors, and users reported that the website was slow.

Step 7 – Investigate the Underlying Cause: I investigated the underlying cause and found that the stress command had

consumed all available CPU resources. The Apache server could not handle incoming requests, and connections were queued and timed out.

Step 8 – Record the Timeline of Events: I recorded the incident timeline. At 10:00 AM, the CloudWatch alarm triggered. At 10:01 AM, the website became slow. At 10:02 AM, users reported the issue. At 10:05 AM, the investigation started. At 10:10 AM, the root cause was identified. At 10:15 AM, corrective action was applied. At 10:20 AM, the website recovered. At 10:25 AM, the incident was closed.

Step 9 – Apply an Appropriate Corrective Action: The corrective action involved killing the stress process and restarting Apache. I ran `sudo pkill stress` to stop the stress process and `sudo systemctl restart apache2` to restart the web server.

Step 10 – Verify Recovery: I verified that the recovery was successful. CPU usage dropped to 25%, response time returned to 200ms, HTTP status returned to 200, and the website became accessible again.

Step 11 – Document the Root Cause: I documented the root cause as the stress command consuming all CPU resources, making the application unresponsive.

Step 12 – Identify What Information Was Missing During the Investigation: During the investigation, I found that memory metrics, disk I/O metrics, and centralized application logs were missing. These would have helped with faster debugging.

Step 13 – Recommend Additional Monitoring or Operational Controls: I recommended adding memory monitoring, disk monitoring, centralized application logging,

Auto Scaling, and Multi-AZ deployment to improve the system's resilience.

Step 14 – Create an Incident-Response Runbook: I created an incident-response runbook covering detection, investigation, resolution, and post-incident steps. This ensures the team is prepared for future incidents.

Step 15 – Present the Incident from Detection Through Resolution: I presented the incident from detection through resolution in a structured incident report, including the summary, timeline, root cause, corrective actions, and recommendations.

3. Key AWS Concepts Learned

- **CloudWatch Monitoring:** I learned how to create CloudWatch alarms to monitor EC2 metrics like CPU utilization and trigger notifications when thresholds are exceeded.
- **SNS Notifications:** I understood how to use Amazon SNS to send email alerts when CloudWatch alarms fire, ensuring timely notifications.
- **Incident Response:** I gained experience in responding to incidents, from detection to resolution, including investigation and root cause analysis.
- **Monitoring Indicators:** I learned which server indicators are useful for monitoring application health, including CPU, memory, disk, and HTTP status.
- **Runbook Creation:** I learned how to create an incident-response runbook that documents procedures for detecting, investigating, and resolving incidents.

- **Root Cause Analysis:** I practiced identifying and documenting the root cause of an incident.
 - **Recovery Verification:** I understood the importance of verifying recovery by checking key indicators after applying corrective actions.
-

4. Challenges Faced

- **Simulating a Realistic Failure:** The biggest challenge was creating a controlled test condition that realistically simulated an application failure. The stress command worked well for CPU spikes, but other types of failures might require different approaches.
 - **Interpreting Logs:** Interpreting the available logs and indicators to identify the root cause was challenging because I had to correlate information from multiple sources.
 - **Timeline Recording:** Recording the exact timeline of events was challenging because I had to be precise about when each event occurred.
 - **Missing Information:** Identifying what information was missing during the investigation was difficult because I had to imagine what additional metrics would have helped.
 - **Runbook Creation:** Creating a comprehensive incident-response runbook required careful consideration of all possible steps and actions.
-

5. Solutions Implemented

- **Controlled Test Condition:** I used the stress command to create a controlled test condition that simulated a CPU spike. This allowed me to observe the indicators and practice the incident response process.
- **Indicator Observation:** I observed the available indicators and logs to identify the symptoms of the issue. This helped me practice troubleshooting and root cause analysis.
- **Timeline Recording:** I recorded the timeline of events in detail to document the incident accurately. This included the time of the alarm, the start of the investigation, and the recovery time.
- **Root Cause Documentation:** I documented the root cause clearly, explaining why the incident occurred and how it was resolved.
- **Runbook Creation:** I created an incident-response runbook that covers detection, investigation, resolution, and post-incident steps. This provides a structured approach for handling future incidents.
- **Recommendations:** I recommended additional monitoring and operational controls to prevent similar incidents in the future.

6. Security Considerations

- **Alerting:** The CloudWatch alarm and SNS notifications ensure that the operations team is immediately alerted to issues, reducing response time.

- **Incident Response:** The incident-response runbook ensures that incidents are handled consistently and efficiently.
 - **Monitoring:** Proper monitoring of key indicators helps detect issues early and reduce downtime.
 - **Documentation:** Thorough documentation of incidents helps identify patterns and improve future responses.
 - **Access Control:** Only authorized personnel should have access to monitoring tools and logs to prevent unauthorized changes.
 - **Data Protection:** Logs and monitoring data should be protected to prevent unauthorized access.
-

7. Results and Observations

CloudWatch Alarm Created:

I successfully created a CloudWatch alarm named task17-cpu-alarm that triggers when CPU utilization exceeds 80% for 5 minutes. An SNS topic named task17_alarm_topic was also created to send email notifications.

Incident Simulation:

When the stress command was run, the CloudWatch alarm triggered, and I received an email notification. The CPU usage spiked to 100%, and the website became slow.

Root Cause Identified:

The root cause was identified as the stress command consuming all CPU resources. After killing the stress process and restarting Apache, the website recovered.

Recovery Verified:

After the corrective action, CPU usage dropped to 25%, response time returned to 200ms, and the website became accessible again.

Observations:

The incident response process was effective in detecting, investigating, and resolving the issue. The CloudWatch alarm and SNS notifications provided timely alerts, and the runbook helped guide the investigation and resolution.

8. Possible Improvements for Production

If this system were deployed in a production environment, I would recommend the following improvements:

- **Memory Monitoring:** Add memory monitoring to detect memory leaks and high swap usage.
- **Disk Monitoring:** Add disk monitoring to prevent disk full issues.
- **Centralized Application Logging:** Use AWS CloudWatch Logs or ELK Stack for centralized application logging to enable faster debugging.
- **Auto Scaling:** Implement Auto Scaling to handle load spikes automatically and maintain application performance.
- **Multi-AZ Deployment:** Deploy the application across multiple Availability Zones for high availability.
- **CloudWatch Dashboard:** Create a CloudWatch dashboard to visualize all key metrics in one place.

- **Incident Response Automation:** Automate incident response actions using AWS Systems Manager or AWS Lambda.
-

9. Incident-Response Runbook

markdown

Incident-Response Runbook

1. Detection

- CloudWatch alarm triggers (CPU > 80%)
- User reports website issue
- Monitoring dashboard shows red status

2. Investigation

- Check CloudWatch metrics
- SSH into EC2 instance
- Check running processes: `top`, `ps aux`
- Check Apache logs: `/var/log/apache2/error.log`
- Check system logs: `/var/log/syslog`

3. Resolution

- Identify the offending process (e.g., `stress`)
- Kill the process: `sudo pkill <process>`
- Restart the application: `sudo systemctl restart apache2`

- Verify recovery: `curl http://localhost`

4. Post-Incident

- Update CloudWatch dashboard
 - Document root cause
 - Schedule follow-up meeting
 - Update runbook with new learnings
-

10. Incident Report

markdown

Incident Report: INC-001

Summary

- **Date:** 2026-09-05
- **Time:** 10:00 AM - 10:25 AM
- **Duration:** 25 minutes
- **Impact:** Website unavailable for 20 minutes

Detection

- CloudWatch Alarm: CPU > 80%
- User reported slowness

Symptoms

- CPU: 100%

- Response Time: 5000ms
- HTTP 500 errors

Root Cause

- `stress` command consumed all CPU resources

Resolution

- Killed `stress` process
- Restarted Apache

Recovery

- Time: 10:20 AM
- CPU: 25%
- Website: Accessible

Recommendations

1. Add Memory Monitoring
2. Implement Auto Scaling
3. Set up Centralized Logging
4. Create Runbook for future incidents

Lessons Learned

- Always monitor memory metrics
- Test changes in non-production first

- Document incident responses
-

Conclusion

This task successfully demonstrated how to build a cloud operations monitoring and incident exercise for an EC2-hosted application. By configuring CloudWatch monitoring, simulating an incident, investigating the root cause, applying corrective actions, and documenting the entire process, I created a complete operational workflow for detecting and responding to problems. The key takeaway is that a well-defined incident response process, combined with proper monitoring and runbooks, is essential for maintaining application reliability and minimizing downtime.